

BÀI TẬP 1
PACKET CAPTURE & PARSER CHO HỆ
THỐNG IDS

1. Mục tiêu

Xây dựng module Packet Capture & Parser cho một hệ thống Intrusion Detection System (IDS) đơn giản bằng **Python**. Module phải thu thập lưu lượng mạng từ live traffic hoặc file PCAP, phân tích các protocol được yêu cầu và chuyển packet thành một cấu trúc dữ liệu chuẩn hóa để các module IDS phía sau xử lý mà không cần truy cập trực tiếp raw packet.

Layer	Protocol
Network	IPv4
Transport	TCP, UDP
Application	HTTP/1.x, DNS, SMTP

2. Yêu cầu chức năng

2.1. Packet Capture

Chương trình phải hỗ trợ đồng thời hai chế độ hoạt động:

- Live capture từ một network interface.
- Import và xử lý packet từ file PCAP.

2.1.1. Live Capture

Chương trình phải cho phép chọn network interface và bắt packet trực tiếp. Có thể sử dụng Scapy, PyShark, libpcap wrapper hoặc thư viện Python tương đương.

```
python main.py --interface eth0
python main.py --interface wlan0
```

Module phải ghi nhận được thời điểm packet được thu nhận và chuyển packet ngay vào parsing pipeline.

2.1.2. PCAP Import

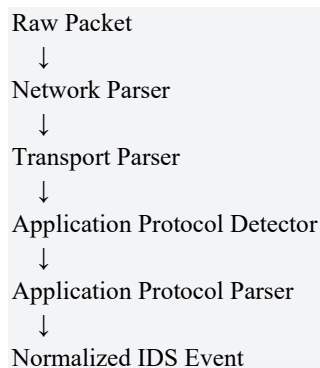
Chương trình phải đọc được packet từ file PCAP và đưa packet vào cùng parsing pipeline với live traffic.

```
python main.py --pcap test.pcap
```

Yêu cầu bắt buộc: không xây dựng hai parser độc lập cho live traffic và PCAP.

3. Packet Parsing và pipeline xử lý

Mỗi packet phải được chuyển thành object/dictionary chuẩn hóa theo pipeline logic sau:



Nếu packet không thuộc các protocol được hỗ trợ, chương trình không được crash. Packet có thể được đánh dấu là UNKNOWN hoặc bỏ qua theo cấu hình.

4. Yêu cầu đối với các parser

Các parser của các giao thức cần lấy được các trường thông tin của các giao thức được hỗ trợ.

5. Nhận diện Application Protocol

Không phụ thuộc tuyệt đối vào port. Port có thể được sử dụng làm tín hiệu nhận diện ban đầu, nhưng sinh viên được khuyến khích kết hợp port-based detection và payload-based detection.

Ví dụ, HTTP request có thể được nhận diện từ các method ở đầu payload như GET, POST, PUT, DELETE, HEAD hoặc OPTIONS. Có điểm thưởng nếu chương trình nhận diện đúng application protocol trên non-standard port.

6. Cấu trúc output chuẩn hóa

Module phải tạo output có cấu trúc thống nhất. Có thể sử dụng Python dictionary, dataclass hoặc JSON-compatible object.

Nguyên tắc thiết kế: Detection Engine ở các bài tập sau không được cần truy cập trực tiếp Scapy/PyShark packet; chỉ làm việc với cấu trúc output chuẩn hóa này.

7. Error Handling

Chương trình không được dừng khi gặp malformed packet, unsupported protocol, missing header, empty payload, truncated PCAP packet hoặc payload không decode được.

8. Logging

Chương trình ghi kết quả parse ra file. Định dạng là JSON Lines (mỗi dòng tương ứng một packet/event).

```

{"packet_id":1,"src_ip":"10.0.0.1","dst_ip":"10.0.0.2",...}
{"packet_id":2,"src_ip":"10.0.0.2","dst_ip":"10.0.0.1",...}
  
```

Output này sẽ được tái sử dụng trong các bài tập IDS tiếp theo.

9. Test cases bắt buộc

Test	Yêu cầu
TCP handshake	Nhận diện SYN, SYN/ACK, ACK
TCP data	Parse TCP packet có payload

Test	Yêu cầu
UDP	Parse UDP packet
HTTP GET	Parse HTTP request
HTTP POST	Parse HTTP request có body
HTTP response	Parse status code và header
DNS Query	Parse domain và query type
DNS Response	Parse ít nhất một answer
SMTP command	Parse HELO/EHLO, MAIL FROM hoặc RCPT TO
SMTP response	Parse SMTP status code
Unknown protocol	Không crash
Malformed packet	Không crash

10. Yêu cầu nộp bài

1. Sử dụng GitHub

- Mỗi sinh viên phải sử dụng **một repository GitHub riêng** cho toàn bộ bài tập. Toàn bộ bài tập lớn chỉ sử dụng 1 đường link github duy nhất này.
- Repository phải được sử dụng ngay từ khi bắt đầu làm bài.
- Project phải để ở chế độ công khai, source code đặt ở nhánh `main`. Nhánh `main` là nhánh mặc định.
- Đặt tên cho repo: **Mã lớp_Họ & tên_MSSV**.

2. Yêu cầu commit

- Sau khi hoàn thành mỗi task, sinh viên phải commit mã nguồn ngay (Task là một công việc nhỏ, độc lập, ví dụ: xây dựng module thu thập packet, trích xuất feature, phát triển dns/http/tcp/udp parser, phát hiện port scan, tạo chức năng cảnh báo...)
- Sau khi thực hiện mỗi test case, sinh viên phải commit kết quả kiểm thử và các thay đổi liên quan. Tạo một thư mục TEST trong project để upload kết quả các testcase.
- Mỗi commit chỉ được chứa nội dung liên quan đến một task hoặc một test case.
- **Không được commit dồn nhiều task hoặc nhiều test case trong một commit.**
- Không được chờ hoàn thành nhiều task rồi mới đưa mã nguồn lên GitHub.

3. Nội dung commit

- Commit message phải ngắn gọn và mô tả đúng công việc đã thực hiện.
- Ví dụ:
 - Implement packet capture
 - Add port scan detection
 - Test port scan - case 01
 - Fix DNS detection false positive

4. Bảo toàn lịch sử

- Không được xóa, sửa hoặc làm lại lịch sử commit để che giấu quá trình phát triển.
- Giảng viên có quyền kiểm tra toàn bộ **Git history** trong quá trình chấm bài.

5. Sử dụng công cụ AI

- Nếu sử dụng AI, sinh viên phải ghi rõ công cụ, mục đích sử dụng và phần mã nguồn có sử dụng AI trong `README.md`.
- Sinh viên phải hiểu và có khả năng giải thích mã nguồn mình đã nộp.

6. Lưu ý trong quá trình làm bài

- Git history, mức độ tương đồng giữa mã nguồn của các sinh viên và khả năng giải thích mã nguồn sẽ được sử dụng để đánh giá tính trung thực của bài làm.
- Giảng viên sẽ thực hiện vấn đáp trong các buổi học.
- Mọi hành vi gian lận hoặc không trung thực sẽ bị 0 điểm quá trình.

— Hết —